



Team TROOS

Software Requirements Specification

Introduction

User Stories

US-01: Capture Consultant Personal Details

US-02: Capture Consultant Skills and Certifications

US-03: Set Consultant Availability Status

US-04: Set Consultant Cost-to-Company Rate

US-05: Consultant profile view

US-06: Create a Project Specification

US-07: Provide POPIA Consent

Functional and Non-Functional Requirements

Functional Requirements

R1: Authentication and Authorisation

R2: Consultant Profile Management

R3: Project Specification Management

R4: Fit Score Matching Engine

R5: Two-Tier Configurable Scoring Algorithm

R6: Placement Recommendation Dashboard

R7: AI-Powered CV Parsing

Non-functional requirements

NFR1: Configurability

NFR2: POPIA Compliance

NFR3: Usability

NFR4: Performance

NFR5: Security

NFR6: Availability

NFR7: Scalability

NFR8: Maintainability

NFR9: Auditability

NFR10: Data Integrity

High-level Use Cases

UC1: Login

UC2: Create a consultant profile

UC3: Upload and parse a CV

UC4: Create a project specification

UC5: Configure firm-wide scoring defaults

UC6: Configure project-level scoring override

UC7: Run a consultant match

UC8: View placement recommendations

UC9: Place a consultant

Architectural Patterns

Layered Pattern

Pattern Description

Layer Structure

Database Layer

[Governing style: Layered \(N-Tier\) Style](#)

[Applied Tactics](#)

[Quality Attributes Quantified](#)

[Justification](#)

[Tradeoffs](#)

[Pipes & Filters Pattern](#)

[Pattern Description](#)

[Governing Style: Event-Driven Style](#)

[Architectural Tactics Applied](#)

[Application in ConsultIQ](#)

[CV Parsing Pipeline \(Sequential\)](#)

[Fit Score Computation Pipeline \(Sequential\)](#)

[Quality Attribute Quantified](#)

[Justification](#)

[Tradeoffs](#)

[Publish-Subscribe Pattern](#)

[Governing Style: Event-Driven Style](#)

[Architectural Tactics Applied](#)

[Quality Attribute Quantified](#)

[Justification](#)

[Tradeoffs](#)

[MVVM – Frontend Architecture](#)

[Pattern Description:](#)

[Governing Style: Layered \(N-Tier\) Style](#)

[Architectural Tactics Applied](#)

[Quality Attributes Quantified](#)

[Justification](#)

[Tradeoffs](#)

[How the Patterns Work Together](#)

[Conclusion](#)

[Design patterns in ConsultIQ](#)

[Strategy pattern](#)

[Data Transfer Object \(DTO\) Pattern](#)

[Chain of responsibility](#)

[Diagrams](#)

[Domain Model](#)

[Use Case Diagrams](#)

Introduction

This Software Requirements Specification (SRS) document details the functional and non-functional requirements for the system being developed for Team TROOS. The primary purpose of the system is to streamline the placement of consultants on client engagements through a centralised and automated platform.

User Stories

US-01: Capture Consultant Personal Details

As a consultant manager, I want to create a consultant profile so that all relevant consultant information is captured in a single workflow.

Acceptance Criteria:

Personal Details

- The form accepts the following: first name, surname, nationality, date of birth and ID number.
- If the consultant is a foreign national, a valid work permit expiration date must be provided
- The system validates that the consultant is older than 15 years
- The system validates that the work permit (if required) has not expired

Contact Details

- The form accepts a phone number and email address
- The phone number must be exactly 10 digits and numeric
- Email must follow a valid format (contain “@” and domain structure)
- Both fields are mandatory

Location Details

- The form accepts a town/city and province
- The province must be selected from a predefined list of South African provinces
- Area code (if included) must be numeric and valid length

Definition of Done:

- All validation rules are covered by automated unit tests

- Code has been peer-reviewed and approved
- No open bugs against this story

US-02: Capture Consultant Skills and Certifications

As a consultant manager, I want to capture a consultant's skills, confidence levels, years of experience, and certifications so that we can accurately match capable consultants to suitable projects.

Acceptance Criteria:

- The consultant manager can add one or more skills to a consultant's profile
- For each skill, the consultant manager must select a confidence level from a predefined set: low, standard and high.
- For each skill, the consultant manager must enter the number of years of experience as a positive whole number
- The consultant manager can upload certified copies of certifications as supporting documents (accepted formats: PDF, JPG, PNG)
- Multiple skills can be added in a single session
- A skill entry cannot be saved without a confidence level and years of experience specified
- Uploaded certification documents are stored and linked to the relevant skill

Definition of Done:

- File upload functionality tested with valid and invalid file types and sizes
- Code has been peer-reviewed and approved
- No open bugs against this story

US-03: Set Consultant Availability Status

As a consultant manager, I want to set a consultant's availability status so that only available consultants are considered during project matching.

Acceptance Criteria:

- A toggle control is displayed on the consultant's profile with two states: Available and Unavailable
- The toggle reflects the consultant's current status when the profile is opened

- Changing the toggle immediately updates the consultant's status in the system
- The updated status is visible to project managers when browsing consultants
- An unavailable consultant does not appear in project matching results.

Definition of Done:

- Toggle state change is covered by automated tests, including edge cases (e.g., rapid toggling)
- Code has been peer-reviewed and approved
- No open bugs against this story

US-04: Set Consultant Cost-to-Company Rate

As a consultant manager, I want to set a consultant's cost-to-company rate so that project budgets can be accurately assessed during the matching process.

Acceptance Criteria:

- The consultant manager can enter an hourly cost-to-company rate for a consultant
- The rate must be a positive numeric value; the form rejects zero or negative values
- The currency is displayed alongside the rate field (Rands is the default).
- The rate is stored against the consultant's profile and visible to project managers during matching
- The consultant manager can update the rate at any time; a history of rate changes is maintained

Definition of Done:

- Rate validation is covered by automated unit tests
- Code has been peer-reviewed and approved
- No open bugs against this story

US-05: Consultant profile view

As a consultant, I want to view my own profile so that I can verify that my information is accurate and request corrections if needed.

Acceptance Criteria:

- The consultant can view their personal details, contact details, location, skills, confidence levels, certifications, and availability status on a single profile page

- The consultant can flag any field as incorrect and submit a correction request with a note explaining the issue
- The consultant cannot directly edit fields that are controlled by consultant manager (e.g. cost-to-company rate, availability status)
- The consultant receives a confirmation message once a correction request has been submitted
- The consultant manager is notified of the correction request for review and action

Definition of Done:

- Correction request workflow tested end-to-end, including consultant manager notification
- Code has been peer-reviewed and approved
- No open bugs against this story

US-06: Create a Project Specification

As a project manager, I want to create a project specification so that the system can match the most suitable consultants to the project.

Acceptance Criteria:

- The project manager can create a new project by entering the following: project name, description, required skills, minimum competency level per skill, location, start date, expected duration, team size, and available client billing budget
- At least one required skill with a minimum competency level must be added before the project can be saved
- A skill must be specified, whether it's mandatory for a project
- Team size must be a positive whole number
- The budget must be a positive numeric value in rands
- Start date cannot be set in the past

Definition of Done:

- All field validations are covered by automated unit tests
- Code has been peer-reviewed and approved
- No open bugs against this story

US-07: Provide POPIA Consent

As a consultant, I want to provide consent for my personal data to be collected and processed so that my profile can be legally stored and used.

Acceptance criteria:

- The consultant must explicitly accept a consent agreement before profile creation
- The consent clearly states:
 - o What data is collected
 - o Why it is collected
 - o How it will be used
- The system prevents profile creation if consent is not given

Definition of done:

- All field validations are covered by automated unit tests
- Code has been peer-reviewed and approved
- No open bugs against this story

Functional and Non-Functional Requirements

Functional Requirements

R1: Authentication and Authorisation

The system must control access based on user roles. Four roles are defined: Admin, Project Manager, Consultant Manager and Consultant. Role assignment is managed by the Admin. Each role has a defined set of permissions that cannot be overridden by the user.

R1.1: User Login

R1.1.1 The system shall allow users to log in securely using their email address and password.

R1.1.2 The system shall redirect the user to their role-appropriate dashboard upon successful login.

R1.1.3 The system shall display a clear error message when login credentials are invalid, without specifying whether the email or password is incorrect.

R1.1.4 The system shall invalidate the user's session after 8 hours of inactivity and require re-authentication.

R1.2: User Logout

R1.2.1 The system shall allow any authenticated user to log out of their active session at any time.

R1.2.2 The system shall invalidate the user's session token immediately upon logout.

R1.3: Role-Based Access Control

R1.3.1 The system shall enforce four distinct user roles: Admin, Project Manager, Consultant Manager and Consultant.

R1.3.2 The system shall restrict firm-wide scoring configuration to Admin users only. Project Manager, Consultant Manager and Consultant shall not access or modify firm-wide scoring defaults.

R1.3.3 The system shall restrict project specification creation, editing, match run execution, and project-level scoring configuration to Project Manager users.

R1.3.4 The system shall restrict consultant profile creation, editing, and CV upload to Consultant Manager users and subject to the Manager's approval, the system shall allow the consultant to edit their profile.

R1.3.5 The system shall restrict the visibility of consultant cost-to-company rates to Admin and Consultant Manager users only. Project Managers shall not see cost-to-company values.

R1.3.6 The system shall return a 403 Forbidden response for any request by a user attempting to access a Project or action outside their role permissions.

R2: Consultant Profile Management

The system maintains a centralised pool of consultant profiles. Each profile captures the data required by the scoring engine to evaluate a consultant against a project. Profiles are created and maintained by Consultant Managers. All authenticated users can view profiles, subject to field-level access controls.

R2.1: Profile Creation

R2.1.1 The system shall allow a Consultant Manager to create a consultant profile capturing the following fields: full name, email address, city and province of residence, and cost-to-company rate (stored in ZAR per hour).

R2.1.2 The system shall allow a Consultant Manager to add one or more skills to a consultant profile. Each skill entry shall capture the skill name (selected from the firm's skills reference list), years of experience as a whole number, and a confidence level on a scale of 1 to 10.

R2.1.3 The system shall automatically derive and store a competency level for each skill entry using a weighted formula applied to years of experience and confidence level. The derived level shall be one of: Beginner, Intermediate, or Expert.

R2.1.4 The system shall display the derived competency level to the Consultant Manager in real time as years of experience and confidence values are entered, before the profile is saved.

R2.1.5 The system shall allow a Consultant Manager to add one or more certifications to a consultant profile, capturing the certification name and issuing body.

R2.1.6 The system shall prevent the same skill from being added more than once to a single consultant profile.

R2.1.7 The system shall save the consultant profile and all associated skills and certifications within a single database transaction. If any part of the save fails, no data shall be persisted.

R2.2: Profile Editing

R2.2.1 The system shall allow a Consultant Manager to edit any field on an existing consultant profile, including personal details, skills, and certifications.

R2.2.2 The system shall recompute and update the competency level for any skill where the years of experience or confidence level is changed during an edit.

R2.2.3 The system shall replace the consultant's full skill set atomically on update. The incoming skill list replaces all existing skill entries in a single transaction.

R2.3: Profile Viewing

R2.3.1 The system shall allow all authenticated users to view a paginated list of all consultant profiles, displaying each consultant's name, location, availability status, and primary skills.

R2.3.2 The system shall allow all authenticated users to view the full detail of an individual consultant profile, including all skills with competency levels, certifications, and availability.

R2.3.3 The system shall display the cost-to-company rate on the profile detail view only to users with Admin or Consultant Manager roles. The field shall not be visible or returned in any API response for Project Manager users.

R2.4: Search and Filtering

R2.4.1 The system shall allow users to search for consultants by name using a text input that performs a case-insensitive partial match.

R2.4.2 The system shall allow users to filter the consultant list by one or more of the following: skill, competency level, city, and availability status. Multiple filters shall be applied simultaneously using AND logic.

R3: Project Specification Management

A project specification defines the requirements a consultant must meet to be placed on a client engagement. Project specifications are the input to the matching engine. They are created and managed by Project Managers.

R3.1: Project Creation

R3.1.1 The system shall allow a Project Manager to create a project specification capturing the following fields: project name, client name, project location (city and province), start date, end date, team size, required consultant allocation as a percentage of full-time capacity, and available client billing budget in ZAR per hour.

R3.1.2 The system shall allow a Project Manager to add one or more skill requirements to a project specification. Each skill requirement shall capture the skill name (selected from the firm's skills reference list), the minimum competency level required (Beginner, Intermediate, or Expert), and a mandatory flag indicating whether possession of this skill is a hard requirement for consideration.

R3.1.3 The system shall prevent the same skill from being added more than once to a single project specification and shall display a clear error if this is attempted.

R3.1.4 The system shall save the project specification and all associated skill requirements within a single database transaction. If any part of the save fails, no data shall be persisted.

R3.1.5 The system shall set the initial status of a newly created project to Open.

R3.2: Project Editing

R3.2.1 The system shall allow a Project Manager to edit any field on an existing project specification.

R3.2.2 The system shall allow a Project Manager to add, edit, or remove individual skill requirements from an existing project specification.

R3.3: Project Viewing

R3.3.1 The system shall allow all authenticated users to view a list of all project specifications, displaying the project name, client name, location, status, start date, and number of required skills.

R3.3.2 The system shall allow all authenticated users to view the full detail of an individual project specification, including all skill requirements, minimum competency levels, and mandatory flags.

R3.4: Project Status Management

R3.4.1 The system shall support the following project statuses: Open, In Progress, and Closed.

R3.4.2 The system shall allow a Project Manager to manually update a project's status through the project edit interface.

R3.4.3 The system shall restrict match run execution to projects with a status of Open. Attempting to run a match against an In Progress or Closed project shall return a clear error to the user.

R4: Fit Score Matching Engine

The matching engine computes a weighted fit score for every consultant in the pool against a selected project. The score is a number between 0 and 100, where a higher score indicates a better fit. The engine incorporates five factors, each contributing a weighted sub-score. The weights and factor inclusions are determined by the active scoring configuration at the time of the match run.

R4.1: Match Execution

R4.1.1 The system shall allow a Project Manager to trigger a match run for any project with a status of 'Open'.

R4.1.2 The system shall compute a fit score for every consultant in the pool against the selected project and return a ranked list ordered by fit score, highest first.

R4.1.3 The system shall incorporate the following five factors in the fit score computation: skill alignment, competency level match, availability, cost fit, and geographic fit.

R4.1.4 The system shall apply the project-level scoring configuration for match runs where a project-level override exists and the firm-wide default configuration otherwise.

R4.1.5 The system shall store each match run result with a timestamp and the scoring configuration snapshot used to support auditability of placement recommendations.

R4.2: Skill Alignment

R4.2.1 The system shall compute the skill alignment sub-score as the proportion of the project's required skills possessed by the consultant, expressed as a percentage.

R4.2.2 The system shall exclude any consultant from the ranked results who does not possess at least one skill flagged as mandatory on the project specification, unless hard exclusion is disabled in the active scoring configuration.

R4.3: Competency Match

R4.3.1 The system shall compute the competency match sub-score by evaluating the consultant's competency level against the minimum required level for each required skill.

R4.3.2 The system shall award full marks on the competency sub-score for any skill where the consultant's competency level meets or exceeds the minimum required level.

R4.3.3 The system shall award partial marks on the competency sub-score for any required skill where the consultant's competency level is below the minimum required level. Partial marks are calculated proportionally based on the distance from the required level.

R4.3.4 The system shall award zero marks on the competency sub-score for any required skill that the consultant does not possess.

R4.4: Availability

R4.4.1 The system shall compute the availability sub-score based on the consultant's remaining capacity during the project period, derived from their active placement records.

R4.4.2 The system shall award full marks on the availability sub-score where the consultant's available capacity meets or exceeds the project's required allocation percentage.

R4.4.3 The system shall award partial marks on the availability sub-score where the consultant's available capacity is greater than zero but below the required allocation percentage.

R4.4.4 The system shall award zero marks on the availability sub-score where the consultant has no remaining capacity during the project period.

R4.5: Cost Fit

R4.5.1 The system shall compute the cost fit sub-score by comparing the consultant's cost-to-company rate against the project's available billing budget per hour.

R4.5.2 The system shall award full marks on the cost fit sub-score where the consultant's cost-to-company rate is less than or equal to the project's billing budget.

R4.5.3 The system shall award zero marks on the cost fit sub-score where the consultant's cost-to-company rate exceeds the project's billing budget.

R4.6: Geographic Fit

R4.6.1 The system shall compute the geographic fit sub-score based on the proximity of the consultant's city of residence to the project location, using a region-based proximity model.

R4.6.2 The system shall award full marks on the geographic fit sub-score where the consultant and project are in the same city.

R4.6.3 The system shall award partial marks on the geographic fit sub-score where the consultant and project are in the same province but different cities.

R4.6.4 The system shall award reduced marks on the geographic fit sub-score where the consultant and project are in different provinces.

R5: Two-Tier Configurable Scoring Algorithm

The scoring engine must not contain any hardcoded business logic. All factor weights, factor inclusions, and hard exclusion settings must be stored in the database and configurable through the admin interface. The configuration operates at two levels: a firm-wide default and a project-level override. Project-level settings always take precedence over firm-wide defaults for that match run.

R5.1: Firm-Wide Scoring Configuration

R5.1.1 The system shall allow an Admin to configure firm-wide default scoring factor weights through a dedicated configuration interface.

R5.1.2 The system shall allow an Admin to activate or deactivate individual scoring factors at the firm-wide level. Deactivated factors are excluded from all match run computations using the firm-wide default.

R5.1.3 The system shall validate that the sum of all active scoring factor weights equals exactly 100% and shall reject the configuration with a clear error message if validation fails.

R5.1.4 The system shall seed the firm-wide scoring configuration on initial setup with the following default weights (subject to change): Skill Alignment 40%, Competency Match 30%, Availability 15%, Cost Fit 10%, and Geographic Fit 5%, with all five factors active and hard exclusion enabled.

R5.1.5 The system shall record a timestamped audit log entry every time the firm-wide scoring configuration is changed, capturing the previous values, the new values, and the identity of the Admin who made the change.

R5.2: Project-Level Scoring Override

R5.2.1 The system shall allow a Project Manager to configure a project-level scoring override for a specific project from the project detail view.

R5.2.2 The system shall allow a Project Manager to activate, deactivate, or reweight individual scoring factors at the project level. Only factors being explicitly changed need to be stored; all other factors inherit their values from the firm-wide default.

R5.2.3 The system shall apply the project-level scoring configuration in place of the firm-wide default for all match runs executed against that project.

R5.2.4 The system shall fall back to the firm-wide default value for any scoring factor not explicitly overridden at the project level.

R5.2.5 The system shall validate that the sum of all active factor weights in a project-level override equals 100% before saving.

R6: Placement Recommendation Dashboard

The dashboard is the primary output of the system. It presents the results of a match run in a format that is explainable and auditable – a manager must be able to understand exactly why a consultant ranked where they did without technical knowledge of the scoring algorithm.

R6.1: Results Display

R6.1.1 The system shall display match run results as a ranked list of consultants ordered by total fit score, from highest to lowest.

R6.1.2 The system shall display the total fit score for each consultant as a number between 0 and 100.

R6.1.3 The system shall display a per-factor score breakdown for each consultant in the ranked list, showing the sub-score and the weight applied for each active scoring factor.

R6.1.4 The system shall display each consultant's name, city, current availability, and their competency level for each matched skill alongside their score.

R6.1.5 The system shall display the scoring configuration used for the match run, including factor weights, so that the basis for the ranking is transparent.

R6.2: Explainability

R6.2.1 The system shall provide a plain-language explanation for each consultant's ranking, describing the primary reasons they ranked where they did relative to the project requirements.

R6.2.2 The system shall clearly indicate for each consultant which mandatory skills they possess and which they are missing.

R6.2.3 The system shall indicate what changes to a consultant's profile or the project configuration would be required for a lower-ranked consultant to improve their position.

R6.3: Placement Action

R6.3.1 The system shall allow a Project Manager to mark a consultant as placed on a project directly from the recommendation dashboard.

R6.3.2 The system shall create a placement record when a consultant is marked as placed, capturing the consultant identifier, project identifier, start date, planned end date, and allocation percentage.

R6.3.3 The system shall immediately reduce the placed consultant's available capacity to reflect the new placement, affecting all subsequent match runs for overlapping project periods.

R7: AI-Powered CV Parsing

CV parsing allows a Consultant Manager to upload a consultant's CV and have the system automatically extract structured data from it using an LLM. The extracted data is always presented for human review before being applied to the profile. No LLM output is ever written directly to a profile without explicit manager confirmation.

R7.1: File Upload

R7.1.1 The system shall allow a Consultant Manager to upload a consultant's CV in PDF format from the consultant profile creation or edit screen.

R7.1.2 The system shall store the uploaded CV as binary data in the database, associated with the consultant's record, retaining the original filename and MIME type.

R7.1.3 The system shall reject uploads that exceed 10MB or are not in PDF format, displaying a clear error message to the user.

R7.2: Extraction and Review

R7.2.1 The system shall submit the uploaded CV to an LLM-based extraction service to identify skills, years of experience per skill, competency signals, certifications, and total years of professional experience.

R7.2.2 The system shall present all extracted data to the Consultant Manager in a structured review screen before any changes are applied to the consultant's profile.

R7.2.3 The system shall display the LLM's reasoning for each inferred competency level alongside the extracted skill data in the review screen, allowing the manager to understand and evaluate the inference.

R7.2.4 The system shall allow the Consultant Manager to accept, edit, or remove any individual extracted item during the review step.

R7.2.5 The system shall apply the reviewed and confirmed extraction data to the consultant's profile only after the Consultant Manager explicitly confirms the submission. Closing or cancelling the review screen shall discard all extracted data without modifying the profile.

R7.2.6 The system shall flag any skill extracted from the CV that does not match an existing entry in the firm's skills reference list. Flagged skills shall be presented separately in the review screen and shall require an Admin to approve their addition to the reference list before they can be saved to the consultant profile.

R7.2.7 The system shall apply the competency derivation formula to any skill accepted from the extraction, computing the competency level from the extracted years of experience and confidence signal before saving to the profile.

Non-functional requirements

NFR1: Configurability

No consultancy-specific business logic may be hardcoded in the application. All placement rules, scoring factors, factor weights, hard exclusion settings, and competency derivation Consultant thresholds must be configurable through the admin interface and stored in the database. A developer should never need to deploy a code change to modify how the scoring engine evaluates consultants. Violations of this requirement shall be treated as critical defects and shall be rejected at code review.

NFR2: POPIA Compliance

All personal consultant data - including full name, email address, location, cost-to-company rate, skills, certifications, and CV files - must be handled in compliance with the Protection of Personal Information Act (POPIA). Specifically:

Data storage: all personal data must be stored in a hosting environment within the Republic of South Africa or in a jurisdiction approved under POPIA.

Access controls: cost-to-company rate and CV files must only be accessible to users with Admin or Consultant Manager roles, enforced at both the API and database levels.

Consent: the system must document the basis on which consultant personal data is collected and processed.

Retention: a data retention policy must be defined and documented, specifying how long consultant data is retained and the process for data deletion upon request.

NFR3: Usability

The user interface must be operable by users with no technical background. Specifically: the scoring configuration interface must express factor weights as percentages with slider controls rather than raw numeric inputs; the placement recommendation dashboard must communicate scores in plain language alongside numeric values; all form validation errors must be expressed in plain language that describes what the user needs to do to resolve the issue, not what failed internally.

NFR4: Performance

The system shall return match run results for a consultant pool of up to 200 profiles within 5 seconds under normal operating conditions. All other standard user interactions – loading a consultant list, saving a profile, viewing a project – shall be completed within 3 seconds under normal network conditions. These consultant hold times apply to the server-side processing time, excluding network latency on the client side.

NFR5: Security

All data transmitted between the client and server shall be encrypted using TLS 1.2 or higher. Sensitive fields at rest – including cost-to-company rate and CV file content – shall be encrypted in the database. Session tokens shall be stored in HTTP-only cookies to prevent client-side script access. All API endpoints shall validate the authenticated user's role before processing any request, server-side, regardless of client-side access controls.

NFR6: Availability

The system shall maintain an uptime of at least 99% during business hours (07h00 to 20h00 SAST, Monday to Friday). Planned maintenance windows must be scheduled outside these hours and communicated to users in advance.

NFR7: Scalability

The system architecture shall support a minimum of 500 consultant profiles and 100 concurrent open projects without response times exceeding the Consultant roles are

defined in NFR4. The data model shall include tenant isolation fields from day one to support multi-tenancy in a future phase without requiring a schema migration.

NFR8: Maintainability

All architectural decisions shall be documented in Architecture Decision Records, each capturing the decision made, the context in which it was made, the alternatives considered, and the tradeoffs accepted. The codebase shall conform to the agreed coding standards document. Every module boundary shall be enforced: no module may import a repository from another module. These rules shall be verified at code review.

NFR9: Auditability

The system shall maintain a tamper-evident audit log of the following events: firm-wide scoring configuration changes (capturing before and after values and the acting user), placement actions (capturing the consultant, project, acting user, and timestamp), and CV extraction events (capturing which CV was processed, what was extracted, and what the manager confirmed or modified). Audit log entries shall never be deleted.

NFR10: Data Integrity

All write operations that involve more than one table - including consultant profile creation with skills, project creation with skill requirements, and CV extraction confirmation - shall be executed within a database transaction. If any step in the operation fails, the entire transaction shall be rolled back. No partial writes shall be persisted under any circumstances.

High-level Use Cases

TUCBW = The use case begins when.

TUCEW = The use case ends when.

UC1: Login

TUCBW: The user navigates to the ConsultIQ login page and enters their email address and password, then clicks Log In.

TUCEW: The user is authenticated and redirected to their role-appropriate dashboard. An active session is established.

UC2: Create a consultant profile

TUCBW, a consultant manager, clicks 'Create Profile' on the consultant list page and begins entering consultant details.

TUCEW, the consultant manager, has entered all required fields, added at least one skill with years of experience and a confidence level, reviewed the derived competency levels, and clicked Save. The consultant profile is visible in the consultant list.

UC3: Upload and parse a CV

TUCBW, a consultant manager, clicks 'Upload CV' on a consultant's profile page and selects a PDF file from their device.

TUCEW, the consultant manager, has reviewed all extracted skills, competency inferences, and certifications; made any necessary corrections; and confirmed the submission, and the consultant's profile has been updated to reflect the confirmed data.

UC4: Create a project specification

TUCBW, a project manager, clicks 'Create Project' on the project list page and begins entering project details.

TUCEW, the project manager, has entered all required project fields, added at least one skill requirement with a minimum competency level and mandatory flag, and clicked Save. The project is visible in the project list with a status of Open.

UC5: Configure firm-wide scoring defaults

TUCBW, an admin navigates to the Scoring Configuration page and views the current firm-wide default factor weights.

TUCEW, the admin has adjusted one or more factor weights or toggled a factor's active state; confirmed that the weights sum to 100%; and saved the configuration. An audit log entry has been created capturing the change.

UC6: Configure project-level scoring override

TUCBW, a project manager, opens a project specification and clicks 'Configure Scoring Override'.

TUCEW, the project manager, has set one or more project-level factor weight overrides, confirmed that the active weights sum to 100%, and saved the override. Subsequent match runs for this project will use the project-level configuration.

UC7: Run a consultant match

TUCBW, a project manager opens an open project specification and clicks Run Match.

TUCEW, the system has computed a weighted fit score for every consultant in the pool against the selected project and returned a ranked list of results on the placement recommendation dashboard.

UC8: View placement recommendations

TUCBW, a consultant manager, views the ranked results on the placement recommendation dashboard following a match run.

TUCEW, the consultant manager, has reviewed the ranked list, examined the per-factor score breakdown and plain-language explanation for one or more consultants, and understands the basis for the ranking.

UC9: Place a consultant

TUCBW, a consultant manager selects a consultant from the ranked results on the placement recommendation dashboard and clicks 'Place'.

TUCEW, a placement record has been created capturing the consultant, project, start date, planned end date, and allocation percentage. The consultant's available capacity has been updated immediately to reflect the new commitment.

Architectural Patterns

Layered Pattern

Pattern Description

This serves as the primary architectural pattern for ConsultIQ. It features a clear separation of concerns across the system's layers: presentation, business logic, and data. This decoupling allows the different layers of our system to scale independently and perform consistently under high-volume loads. The frontend is hosted on Vercel, while the backend is hosted on AWS using EC2 and RDS. This approach enables horizontal scaling across the entire stack.

Layer Structure

Presentation Layer

This tier serves as the primary gateway for users to interact with the platform. This layer communicates with the Business Logic Layer via REST API calls and receives no direct database access to increase security.

Business Layer

The core services for ConsultIQ are found at this tier, which houses all domain rules, algorithms, and process orchestration. This layer operates as a strict mediator between the presentation layer and the data access layer. It propagates requests downward from the user interface and replies upwards to the user interface, applies the necessary business rules, and interfaces with the data layer to persist or retrieve information, ensuring that no database-specific logic leaks into the application's core.

Data Access Layer

This layer abstracts all database access by implementing the repository pattern, ensuring that no database-specific logic ever leaks upward into the Business Logic Layer. This layer will serve as a strictly controlled chokepoint for POPIA compliance to ensure that important user information is not leaked, thus increasing overall security.

Database Layer

Persistence tier where the database is located. This layer interacts with the data access layer, serving as the long-term storage engine for the system.

Governing style: Layered (N-Tier) Style

This architectural style provides a structural skeleton for the backend and organizes the system horizontally into four layers, assigning each layer a strictly defined responsibility. Communication is strictly sequential; each layer only interacts with the one immediately adjacent to it, with requests flowing down the stack and responses returning up the stack. Ultimately, this structure enforces separation of concerns, enhancing maintainability and security across the entire application.

Applied Tactics

- **Isolate responsibilities and depend on abstractions (responsibility localisation & contract-based):** By strictly separating the UI, business rules, and data access, we localise changes and depend on interfaces rather than concrete implementations. This directly addresses the quality attributes of maintainability and testability.
- **Limit Access (Chokepoint):** We force all data retrieval to pass exclusively through the Data Access Layer. This structurally guarantees security requirements by ensuring POPIA compliance rules cannot be bypassed by higher-level code.
- **Resource Re-use (Connection Pooling & Caching):** Because the data access layer acts as a strict chokepoint, it risks becoming a performance bottleneck. To mitigate this, we apply resource-reuse tactics such as TypeORM's built-in connection pool (with a specified maximum pool size) and in-memory caching via the NestJS cache manager (backed by Redis). This directly addresses performance requirements, ensuring the system meets its quantified response time thresholds under heavy load.

Quality Attributes Quantified

- **Maintainability:** Code modifications and feature enhancements are strictly localized due to contract-based communication between tiers. This architectural boundary ensures a 0% ripple effect into non-adjacent layers when updating internal logic or swapping out underlying technologies, which drastically reduces regression testing overhead and safely allows independent layer updates.

- **Security:** While authentication and business-level authorisation are handled higher in the stack, the Data Access Layer serves as a critical structural safeguard for data persistence. By forcing 100% of database queries through a centralised repository pattern chokepoint, it prevents rogue or accidental direct database access from the presentation layer and ensures consistent application of POPIA-compliant auditing and data masking.
- **Testability: Ensures** a minimum of 80% unit test coverage per layer. The Business Logic Layer is testable using mocked repositories, the Presentation Layer with mocked API responses, and the Data Access Layer with an in-memory database.
- **Performance:** The active use of TypeORM connection pooling and Redis caching prevents network bottlenecks, ensuring database query response times remain under 2 seconds even during peak concurrent loads.

Justification

ConsultIQ serves multiple user roles (Consultants, Project Managers, Consultant Managers, and System Admins). Each role interacts with different parts of the system. A layered architecture provides clear boundaries that map directly onto these roles and allow the system to be developed, tested, and maintained in each layer independently.

Tradeoffs

Consideration	Advantage	Tradeoff
Layer Isolation	A bug in the scoring algorithm is confined to the business logic layer and cannot corrupt the data layer or break the UI independently.	Every new end-to-end feature requires coordinated work across all four layers, increasing the per-feature development effort.
POPIA Structural Compliance	The Data Access Layer centralises data access into a single, controllable point, significantly reducing the surface area for compliance violations. This structured pipeline makes it easier to implement consistent auditing, logging, and data masking	The Data Access Layer becomes a bottleneck if not designed with connection pooling and caching; performance must be actively managed.

	required by POPIA, rather than scattering data access throughout the codebase.	
Independent Testability	Each layer can achieve high unit test coverage without the other layers present, enabling fast feedback loops during development.	End-to-end integration tests are still required and more complex to set up, as they must span all four layers simultaneously.
Team Parallelism	Frontend and backend teams can develop simultaneously against agreed REST API contracts.	API contracts must be defined upfront, requiring early design alignment before either side can proceed independently.

Pipes & Filters Pattern

Pattern Description

This pattern structures the system into a sequence of independent components (filters) connected by a standardised data channel (pipes). Each filter performs a mathematical or structural transformation on the input, and it passes the result to the next filter. Filters are independent, as they do not know what comes before or after them in the pipeline

Governing Style: Event-Driven Style

This architectural style defines the overarching rules for cross-component communication within the system. Unlike the layered style, which enforces strict vertical structural boundaries, the event-driven style dictates that distinct system domains must communicate asynchronously by producing and consuming events rather than calling each other directly. A core rule of this style is strict decoupling: producers and consumers must have no direct knowledge of one another. By governing the architecture this way, we structurally ensure that computationally heavy workflows (such as LLM-based CV extraction or multi-factor fit score recalculation) operate completely in the background without blocking the main application threads. This overarching philosophy fundamentally guarantees system responsiveness and protects the application from cascading localized failures

Architectural Tactics Applied

Specific architectural tactics within this pattern have been engineered to meet our required quality attributes.

- **Abstract Processing Stages:** By encapsulating each stage of a multi-step workflow as an independent, replaceable unit, we prevent the scoring and parsing engines from becoming tangled monoliths. This directly addresses the quality attributes such as modifiability and testability.
- **Defer Processing (Asynchronous Pipeline Execution):** Sequential pipelines are computationally heavy. To guarantee system performance, we apply the tactic of deferring processing. The entire sequential pipeline is triggered asynchronously via the Event Bus. This ensures that the heavy, multi-step calculations happen on background workers, returning control to the user interface immediately and preserving system responsiveness.

Application in ConsultIQ

CV Parsing Pipeline (Sequential)

When a consultant's CV is uploaded, it passes through a sequential pipeline before the consultant's profile is updated. The pipeline has two entry points, automated (CV upload) and manual (direct data entry), both converging before profile update.

WHEN A CV_{UPLOADED} EVENT IS CONSUMED, THE FOLLOWING FILTER PIPELINE EXECUTES:

Filter	Name	Transformation
F-1	OCR Extraction	Converts the raw PDF or photo to a structured text string.
F-2	LLM Extraction	Sends the text to an LLM; receives structured JSON of skills, certifications, experience, and competency signals.
F-3	Data staging	Maps the JSON payload into a standardised draft schema and pauses pipeline execution. Acts as a gate until the Consultant Manager confirms or corrects the extracted data.
F-4	Profile Update	Writes the validated, corrected data to the Consultant Profile via the Repository; publishes a ProfileUpdated event.

Each filter in the automated path performs one transformation: OCR converts the raw file to text, LLM structures that text, and the final filter stages the data into a draft state. Because

each filter depends directly on the output of the previous stage, this automated extraction pipeline executes strictly sequentially. Once the pipeline completes, the system transitions to a manual workflow, presenting the staged data to the Consultant Manager for review before the final event to update the profile is published.

Fit Score Computation Pipeline (Sequential)

When the Scoring Engine evaluates a consultant against a project, the computation is structured as an exhaustive, sequential filter pipeline. A consultant's profile enters the pipe, and each filter computes one scoring dimension, appending its result to the data payload before passing it to the next filter. To ensure complete data visibility for management, the pipeline does not short-circuit; every active dimension is evaluated even if a previous filter yields a 0% match. Once the payload passes through F-1 to F-5 sequentially, the final filter (F-6) aggregates all appended data into a single composite fit score.

Filter	Scoring Dimension	What It Computes
F-1	Skill Alignment	Percentage match between consultant skills and required project skills.
F-2	Competency Level	Whether consultant proficiency meets minimum project thresholds per skill.
F-3	Geographic Feasibility	Travel cost and time from consultant location to project site via Mapping API.
F-4	Cost vs Billing	Ratio of consultant cost-to-company against project billing budget.
F-5	Availability	Whether consultant is available for the project's start date and duration.
F-6	Weight Aggregation	Applies consultancy-level or project-level weights; produces the final composite fit score.

Filters can be activated or deactivated per project via the two-tier configuration system. A local project might deactivate Filter 3 (geographic) entirely, while a specialised engagement might assign disproportionate weight to Filter 2 (competency).

Quality Attribute Quantified

- **Performance:** By applying the Defer Processing tactic, the exhaustive pipeline is offloaded to background event consumers. This guarantees the UI remains highly

responsive, consistently meeting a response time threshold of under 3 seconds even while the backend calculates complex composite scores for a pool of 200+ consultants.

- **Modifiability:** Adding a new scoring dimension requires writing a single new filter and registering it, resulting in zero modifications (0 lines of code changed) to any existing filters or the final aggregation logic.
- **Testability:** Because each filter acts as an isolated processing unit, mock dependencies can be injected, enabling >85% unit test coverage for the scoring and parsing logic in isolation.

Justification

Both the CV Parsing process and the Fit Score computation are inherently sequential, transformational workflows. The Pipes & Filters pattern is a natural architectural fit because it enforces strict single-responsibility principles. By isolating each processing stage, we ensure that individual filters can be rigorously tested, modified, or replaced without affecting adjacent stages or hard-coding complex business logic into a single monolithic function.

Tradeoffs

Consideration	Advantage	Tradeoff
Modularity	Each filter is a self-contained unit, making the pipeline easy to extend and modify	Defining clean filter boundaries requires careful upfront design to avoid filters carrying more load than necessary
Testability	Each filter can be unit tested in complete isolation from the rest of the pipeline	End-to-end pipeline tests are still required to catch integration issues between filters
Error Handling	A failed filter can be isolated and retried without restarting the pipeline	Partial pipeline failures require state management to know which stage failed and resume correctly

Performance	Filters can potentially be parallelised for independent scoring dimensions	Sequential pipelines add latency overhead compared to a single monolithic computation function
--------------------	--	--

Publish-Subscribe Pattern

This pattern implements asynchronous communication by using a message bus as an intermediary between components. A producer (publisher) emits a named event to the bus when a meaningful state change occurs, without needing to know which consumers (subscribers) exist. Subscribers register interest in specific events and react independently.

Governing Style: Event-Driven Style

This pattern is the concrete implementation of the Event-Driven Style. It dictates how distinct domains within ConsultIQ communicate globally, ensuring that cross-domain workflows remain decoupled and asynchronous.

Architectural Tactics Applied

We apply specific architectural tactics within this pattern to engineer our performance and modifiability targets.

- **Defer Processing via Event Bus (Queuing):** We explicitly offload long-running, intensive computations from the HTTP request thread. By placing the request in a queue (the event bus) and returning control to the user immediately, to structurally guarantee system responsiveness. This addresses both performance and responsiveness of the platform.
- **Abstract Dependencies (Decoupling):** By forcing components to communicate via events rather than direct method calls, we sever the hard dependencies between domains. This increases the modifiability of the system.

SYSTEM EVENTS:

Event	Triggered By	Consumed By
-------	--------------	-------------

CVUploaded	Manager uploads CV file	CV Parsing Pipeline — begins OCR extraction
ProfileUpdated	CV parsing or ManualEntry completes	Scoring Engine — invalidates and recalculates existing scores
ProjectCreated	The Project Manager defines a new project	Matching Engine — triggers an initial placement run
ConsultantAssigned	Placement is confirmed by a manager	Analytics Service — updates utilisation and placement history
SkillGapDetected	The scoring engine finds no adequate match	Notification Service — alerts the relevant manager with a gap report
ScoringConfigUpdated	Admin changes consultancy-level weights	Scoring Engine — triggers re-evaluation of active projects

Quality Attribute Quantified

- **Performance:** The background worker queues allow the Scoring Engine to concurrently process up to 100 simultaneous ProjectsCreated events without degrading or blocking the HTTP layer.
- **Responsiveness:** By deferring processing, the dashboard returns control to the user immediately, maintaining a UI interaction response time of under 1 second after an event trigger (like a CV upload).
- **Modifiability:** Extensibility is maximised; new services can be added as subscribers with zero modifications to the existing producer codebases.

Justification

ConsultIQ relies heavily on computationally expensive workflows, such as LLM-based CV extraction and multi-factor fit score recalculation. A synchronous approach would force the application to freeze while waiting for these sequential tasks to finish. By employing the Publish-Subscribe bus to defer processing, a component can immediately broadcast a state

change and free up system resources. This prevents localised processing delays from cascading and crashing the surrounding system.

Tradeoffs

Consideration	Advantage	Tradeoff
Decoupling	Components are completely decoupled, making it easier to add new features without affecting existing ones	The overall system flow is no longer linear, making it harder to trace a request end-to-end during debugging
Scalability	Event consumers can be scaled independently based on load	Event infrastructure (queue or bus) adds operational complexity that must be managed
Resilience	A failed consumer does not block the producer — events can be retried independently	Eventual consistency means the dashboard may briefly show stale scores after a profile update
Responsiveness	Processes can be deferred to a background queue. The user interface is never blocked, allowing users to continue working under heavy computations	Immediate consistency is sacrificed. Data displayed to a user might be stale. UI indicators need to be implemented to manage user expectations
Event Cascading	The system automatically maintains global data consistency across the platform when core business rules change.	Global events like ScoringConfigUpdate can trigger massive, simultaneous recalculations across all active projects. This requires mitigation strategies (like batching, throttling, or off-peak scheduled jobs) to prevent system overload and performance degradation.

MVVM – Frontend Architecture

Pattern Description:

MVVM separates the frontend into three layers. The Model represents raw data structures that are returned from the backend API. The view is solely responsible for rendering the UI. The ViewModel sits between the two and fetches data, manages state, handles user actions, and exposes clean binding to the view. In our React implementation, ViewModels are implemented as custom hooks (e.g., `useConsultantProfile`, `useScoringConfig`) that encapsulate API calls, local state, and derived values. The view never calls the API directly and never contains business logic.

Governing Style: Layered (N-Tier) Style

This pattern operates exclusively within the Presentation Layer of the Layered (N-Tier) Style. The ViewModel serves as the *only* point of contact with the N-Tier Business Logic Layer (via REST API). This ensures the frontend remains entirely blind to how scores are computed or data is stored, strictly enforcing the downward-only communication rule of the Layered style.

Architectural Tactics Applied

We apply specific frontend architectural tactics within this pattern to engineer our usability and testability targets.

- **Separate Interface from State:** By delegating all state management, error handling, and API communication to the ViewModel, we force the View to act as a purely stateless rendering component. This structural separation directly addresses component testability and system usability.

Quality Attributes Quantified

- **Testability:** ViewModel hooks can be tested in isolation using React Testing Library's `renderHook`, ensuring the frontend team consistently meets the 80% code coverage threshold for business logic without writing slow, brittle DOM-rendering tests. .

- **Usability:** The strict separation allows UI/UX designers to implement design changes with a 0% risk of regressions to the underlying data-fetching or state-management logic.

Justification

ConsultIQ requires an intuitive UI that users with varying technical expertise (from system admins to consultant managers) can navigate. MVVM traps all the complex state-management logic in the ViewModel layer. Because the Views remain clean and declarative, frontend developers and designers can easily implement progressive disclosure and context-sensitive UI elements without getting tangled in API timing or data-fetching logic

Tradeoffs

Consideration	Advantage	Tradeoff
Separation of Concerns	Views are purely presentational — designers can restyle them without touching any logic	Requires discipline to keep logic out of Views; prop drilling can become verbose for deeply nested components
Testability	ViewModel hooks are independently testable using dedicated hook-testing utilities, speeding up the test cycle without needing full DOM mounts.	More test files are required — one for testing the hook logic in isolation, in addition to component snapshot or interaction tests for the View. .
Reactivity	State changes in the ViewModel automatically propagate to the View through React's rendering model	A complex state with many interdependencies (e.g., scoring weights) requires careful hook design to avoid stale state bugs
Consistency	Every screen follows the same View / ViewModel separation, making the	New team members must understand the hook pattern before contributing, adding a small onboarding cost

	codebase predictable for all team members	
--	---	--

How the Patterns Work Together

The architectural patterns and styles chosen for ConsultIQ do not operate in isolation; they are deliberately layered and complementary. The Layered (N-Tier) Style provides the structural skeleton within which all other decisions operate.

Here is how the hierarchy of decisions interacts to form a cohesive system:

- **The Structural Skeleton:** The Layered Pattern builds the backend, enforcing strict downward-only communication. Within its foundation (the Data Access Layer) , the Repository Pattern acts as a strict POPIA compliance chokepoint.
- **The Frontend Interface:** The MVVM Pattern structures the frontend strictly within the N-Tier Presentation Layer. It abstracts all state and API interactions, communicating exclusively downward with the N-Tier Business Logic Layer.
- **The Internal Processing:** The Pipes & Filters Pattern governs the internal processing logic *within* the Business Logic Layer, turning monolithic scoring tasks into modular, testable stages.
- **The System Communication:** The Event-Driven Style (implemented via the Publish-Subscribe Pattern) governs how these heavy internal components communicate across the system boundary. It allows the strict layers to communicate asynchronously without breaking their structural isolation.

This separation of concerns at the tactical level mirrors the separation of concerns at the structural level, resulting in a system that is consistent from the high-level architecture down to the individual components.

Conclusion

The hybrid architectural approach adopted for ConsultIQ was not chosen based on generic best practices, but because each style, pattern, and tactic addresses a distinct, quantified Quality Requirement.

No single style or pattern was sufficient to satisfy the system's conflicting needs:

- A pure layered style would not elegantly model the sequential nature of CV parsing, nor could it handle score computation synchronously without violating the responsiveness thresholds.
- A pure Pipes & Filters implementation would lack the rigid layered structure required to enforce POPIA data access compliance structurally.
- A pure event-driven system would introduce unnecessary distributed complexity and traceability loss for simple, synchronous CRUD operations.
- Using MVVM without a layered backend would leave the system without a unified source of truth or structural discipline.

By carefully combining these styles to govern the system, these patterns to build the components, and these tactics to manage the behaviour, we have consciously accepted operational complexity in exchange for our core goals. The resulting architecture is POPIA-compliant by structure, responsive under heavy computational load, testable at every layer, and safely extensible for future business requirements.

Design patterns in ConsultIQ

Strategy pattern

ConsultIQ serves multiple user roles that must see different versions of the same data. A consultant manager is authorised to see a consultant's cost-to-company rate, but a project manager must never see it. This is both a business rule and a POPIA-adjacent data protection concern.

The intent behind this design pattern is to define a family of algorithms, encapsulates each one, and make them interchangeable at runtime. The caller selects the behaviour by passing a parameter rather than branching internally with hard-coded logic spread across the codebase.

Rather than duplicating mapping logic per role, the role is passed in as a single parameter and one conditional governs the entire data-shaping strategy. As new roles or visibility rules are introduced, this single decision point is the only place that needs updating, keeping the change surface minima

Example:

```
let sidebarItems;  
if (currentUserRole === "ADMIN") {
```

```
        sidebarItems = adminSidebarItems;
    } else if (currentUserRole === "CONSULTANT_MANAGER") {
        sidebarItems = consultantManagerSidebarItems;
    } else {
        sidebarItems = consultantSidebarItems;
    }
}
```

Data Transfer Object (DTO) Pattern

ConsultIQ's database schema for a consultant is a normalised relational structure. Exposing this raw shape directly through the API would tightly couple the system's API contract to its database schema. Any schema migration would risk breaking the frontend. It would also expose internal fields (like raw foreign keys or audit columns) that users should never see.

The idea behind the pattern is that it carries data between layers using a plain object that contains no behaviour. It decouples the internal database shape from what is exposed to the outside world, giving each layer a stable, purpose-built contract.

DTOs like `ConsultantListItemDto` and `ConsultantProfileDto` act as a translation layer. The mapping happens once inside the service, flattening nested relations, applying null safety, and renaming fields to match the frontend's expectations. The controller and the frontend never touch raw database objects, meaning a database refactor does not cascade into an API break.

```
const mappedProjects: ProjectListItemDto[] = projects.map((p) => ({
    id: p.id,
    projectName: p.projectName,
    clientName: p.clientName,
    city: p.city,
    province: p.province,
    startDate: p.startDate,
    endDate: p.endDate ?? null,
    teamSize: p.teamSize,
    requiredAllocationPercentage: p.requiredAllocationPercentage,
```

```

        clientBillingBudget: Number(p.clientBillingBudget),

        status: p.status,

        skillCount: p.skillCount,

    });

```

Chain of responsibility

Every incoming HTTP request to ConsultIQ must pass several independent checks before reaching the controller: is the user authenticated, do they have the right role, and is the request body shaped correctly? If all of this logic lived inside each controller method, every endpoint would repeat the same boilerplate, and a missed check on a single endpoint would create a security gap.

The goal is to avoid coupling the sender of a request to its receiver. It passes a request along a chain of handlers. Each handler either processes the request and passes it on, or rejects it and stops the chain entirely. No single handler needs to know what comes before or after it.

NestJS guards and pipes form exactly this chain. The AuthGuard checks the JWT token. If it passes, the RoleGuard checks the user's role against the required roles for that endpoint. If that passes, the ValidationPipe checks the request body against the DTO schema. Only if all three pass does the request reach your controller method.

Example of a role guard :

```

@Injectable()

export class RolesGuard implements CanActivate {

    constructor(private readonly reflector: Reflector) {}

    canActivate(context: ExecutionContext): boolean {

        const isPublic =
this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [

            context.getHandler(),

            context.getClass(),

            .

```

```
. // code //

.

    const request = context.switchToHttp().getRequest();

const user = request.user as JwpPayload | undefined;

if (!user) {

    throw new ForbiddenException(

        'Access denied: no authenticated user found on request. ' +

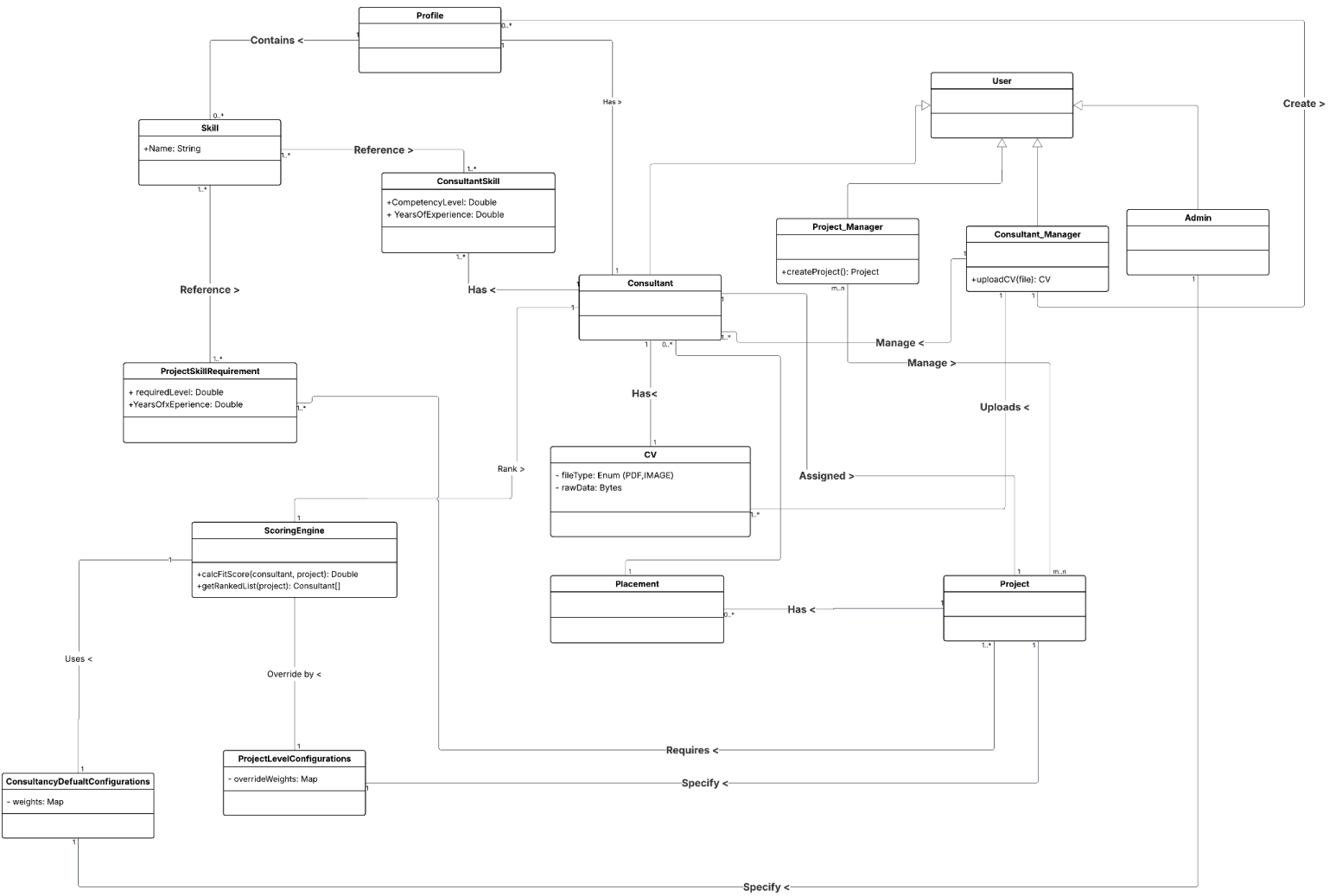
        'Ensure JwpAuthGuard is applied before RolesGuard.',

    );

}
```

Diagrams

Domain Model



Use Case Diagrams

